AI Accelerator Design – Assignment 3

BCHW to B × (CHW) Tensor Flattening and Reconstruction

This assignment is about converting a four-dimensional BCHW tensor into a two-dimensional flattened tensor and then converting it back. The main idea is to understand how the position of every tensor element changes when the data is stored in linear form. This type of conversion is useful in AI accelerators where tensor data can be transferred or buffered in flattened form.

1. Objective

The input tensor has the form I ∈ R^(B×C×H×W), where B is batch size, C is the number of channels, H is height and W is width. The required operations are:

```
B × C × H × W  →  B × (CHW)  →  B × C × H × W
```

The flattening and reconstruction are implemented using loops and explicit index calculations. Direct functions such as reshape(), view(), flatten() and ravel() are not used in the main implementation.

2. Flattening Algorithm

For an element I[b,c,h,w], its position in the flattened row is calculated using:

```
k = c*H*W + h*W + w
```

The value is then copied from I[b,c,h,w] to F[b,k]. The loops go through the batch, channel, height and width dimensions.

3. Reconstruction Algorithm

For reconstruction, the flattened index k is converted back into c, h and w. The calculations used are:

```
c = k // (H*W)
r = k % (H*W)
h = r // W
w = r % W
```

After finding these three indices, the value F[b,k] is placed at I_hat[b,c,h,w].

4. Pseudocode

```
FLATTEN(BCHW):
    read B, C, H, W
    create F with size B x (C*H*W)
    for b:
        for c:
            for h:
                for w:
                    k = c*H*W + h*W + w
                    F[b,k] = I[b,c,h,w]
    return F

RECONSTRUCT(B x CHW):
    create I_hat with size B x C x H x W
    for b:
        for k:
            c = k // (H*W)
            r = k % (H*W)
            h = r // W
            w = r % W
            I_hat[b,c,h,w] = F[b,k]
    return I_hat
```

5. Manual Indexing Example

For the small example B=1, C=2, H=2 and W=3, there are 12 values in one batch. Since H×W = 6, the formula becomes k = c×6 + h×3 + w.

Original index	Calculation	k
[0,0,0,0]	0×6 + 0×3 + 0	0
[0,0,0,1]	0×6 + 0×3 + 1	1
[0,0,1,2]	0×6 + 1×3 + 2	5
[0,1,0,0]	1×6 + 0×3 + 0	6
[0,1,1,2]	1×6 + 1×3 + 2	11

6. Experimental Setup

Synthetic tensors were used for the experiments. This is allowed in the assignment for larger channel counts and can represent intermediate feature maps in a DNN.

Input	B	C	H	W	Emax	MAE
MNIST-like	1	1	28	28	0.0e+00	0.0e+00
CIFAR-like	1	3	32	32	0.0e+00	0.0e+00
Synthetic fmap 1	1	8	16	16	0.0e+00	0.0e+00
Synthetic fmap 2	1	16	16	16	0.0e+00	0.0e+00
Synthetic fmap 3	1	32	16	16	0.0e+00	0.0e+00
Synthetic fmap 4	1	64	16	16	0.0e+00	0.0e+00
Synthetic fmap 5	1	128	8	8	0.0e+00	0.0e+00
Synthetic fmap 6	1	256	8	8	0.0e+00	0.0e+00
Synthetic fmap 7	1	500	8	8	0.0e+00	0.0e+00

7. Error Measures

The reconstruction error is E = I – I_hat. The maximum absolute error is the largest absolute difference between any original and reconstructed element. MAE is the average of all absolute differences.

8. Results and Discussion

The maximum absolute error and MAE were zero for all the tested tensor sizes. This means that the reconstructed tensor was the same as the original tensor. The result also shows that increasing the channel count does not change the indexing method. It only increases the number of values that have to be mapped.

9. Conclusion

The assignment was implemented using explicit indexing and loops. The forward mapping converts BCHW into B×(CHW), and the reverse mapping correctly recovers the original BCHW layout. Since no tensor values are changed during the conversion, the expected reconstruction error is zero, which was observed for all the experiments.

Python file: assignment3_tensor_flatten.py

Note: The experimental tensors are deterministic synthetic tensors used to test the indexing implementation.